RiskLens: Transaction Risk Scoring with Behavioral Patterns and Adaptive Anomaly Identification

A PROJECT BASED LEARNING (PBL) REPORT

Submitted by

SYAMRIT ESWARAN GURUSAMY

S HARSHAVARTHANAN

Submitted in partial fulfillment of the requirements for

the

Project-Based Learning component of Java Programming

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



CHENNAI INSTITUTE OF TECHNOLOGY



(Autonomous)

Affiliated to Anna University, Chennai

October - 2026

Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

IM1. To creates next generation leaders by effective teaching learning methodologies and in still Scientifics park in them to meet the global challenges.

IM2. To transform lives through deployment of emerging technology, novelty and sustainability.

IM3. To inculcate human values and ethical principles to cater the societal needs.

IM4. To contributes towards the research ecosystem by providing a suitable,

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of the Department:

To Excel in the emerging areas of Computer Science and Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to contribute innovatively through advanced computing in real time situations.

Mission of the Department:

DM1. To provide strong fundamentals and technical skills for Computer Science applications through effective teaching learning methodologies.

DM2. To transform lives of the students by nurturing ethical values, creativity and novelty to become Entrepreneurs and establish start-ups.

DM3. To habituate the students to focus on sustainable solutions to improve the quality of life and the welfare of the society.

PROGRAM EDUCATIONAL OBJECTIVES (PEO):

PEO 1: Contribute to the industry as an Engineer through sound knowledge acquired in core engineering to develop new processes and implement the solutions for industrial problems.

PEO 2: Establish an organization / industry as an Entrepreneur with professionalism, leadership quality, teamwork, and ethical values to meet the societal needs.

PEO 3: Create a better future by pursuing higher education / research and develop the sustainable products / solutions to meet the demand.

PROGRAM OUTCOMES (POS):

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOS):

PSO 1 : Analyze, design and develop solutions in the areas of Business Process Management to build the quality products for industry and social needs.

PSO 2 : Innovate ideas and solutions for real time problems in the field of Software Engineering and Mobile applications by adapting the emerging technologies and tools.

CHENNAI INSTITUTE OF TECHNOLOGY, CHENNAI

(Autonomous)
Affiliated to Anna University, Chennai

BONAFIDE CERTIFICATE

This is to certify that the Project–Based Learning report titled " RiskLens: Transaction Risk Scoring with Behavioral Patterns and Adaptive Anomaly Identification " is a Bonafide record of work carried out by SYAMRIT ESWARAN GURUSAMY , 2104251041018 of the Department of Computer Science and Engineering, Chennai Institute of Technology, as part of the continuous, mentor–guided Project-Based Learning (PBL) component of the Java Programming course during the academic year 2026–2027 under my supervision.

SIGNATURE
Dr. S. Pavithra M.E., Ph. D
HEAD OF THE DEPARTMENT
Professor and Head,
Department of Computer Science and Engineering
Chennai Institute of Technology
Chennai - 69

SIGNATURE
Dr.Dharani M.E.,
SUPERVISOR
Designation
Department of Computer Science and Engineering
Chennai Institute of Technology
Chennai - 69

Submitted for the final review held on …….

INTERNAL EXAMINER

DECLARATION

We jointly declare that the PBL report on **" RiskLens: Transaction Risk Scoring with Behavioral Patterns and Adaptive Anomaly Identification "** is the result of original work done by us and best of our knowledge, similar work has not been submitted to **"ANNA UNIVERSITY CHENNAI"** for the requirement of Degree of **BACHELOR OF ENGINEERING**. This PBL report is submitted on the partial fulfillment of the requirement of the award of Degree of **COMPUTER SCIENCE AND ENGINEERING**.

Signature

SIRAGU

SYAMRIT ESWARAN GURUSAMY

S HARSHAVARTHANAN

Place:

Chennai

Date:

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our honourable Chairman **SHRI.P. SRIRAM** for providing immense facilities at our institution.

We are very proudly rendering our thanks to our Principal **Dr.A.RAMESH M.E**, **Ph.D.,** for the facilities and the encouragement given by him to the progress and completion of our project.

We would like to express special thanks of gratitude to our Dean **Dr. V. SRINIVASA RAO, M.E., Ph.D.,** who has been the key spring of motivation to us throughout the completion of our course and project work.

We proudly render our immense gratitude to the Head of the Department **Dr.S.PAVITHRA M.E, Ph.D.,** for her effective leadership, encouragement and guidance in the project.

We would like to extend our thanks to the Project Co-ordinator **Dr.NANDHAKUMAR M.E, Ph.D.,** Department of Computer Science and Engineering, for their valuable suggestions throughout this project.

We wish to acknowledge the help received from the class advisors **Dr.DHARANI M.E,** of the Department of Computer Science and Engineering and others for providing valuable suggestions and for the successful completion of the project.

SYAMRIT ESWARAN GURUSAMY 2104251041018
S HARSHAVARTHANAN 210425104080

ABSTRACT

RiskLens is an educational ATM and digital-banking security simulator developed using **Core Java 21 and the Java Standard Library**. The system extends a conventional banking application with secure authentication, behavioral profiling, weighted rule-based fraud detection, explainable risk scoring, automated security decisions, concurrent event monitoring and tamper-evident audit logging. Each transaction is evaluated using contextual signals such as transaction amount, velocity, time, terminal history, authentication failures, daily limits, recent PIN changes and cross-channel activity. The resulting risk score is mapped to security decisions such as **ALLOW, CHALLENGE, MONITOR and FREEZE**, while the contributing reasons are retained for explain-ability. Controlled demonstrations produced risk levels ranging from **0 to 115**, including a documented 115-point account-freeze scenario. The project demonstrates how object-oriented programming, Java concurrency, collections, cryptography, file handling and design patterns can be integrated into a cyber-security oriented banking application.

Keywords : [Core Java 21, Object-Oriented Programming, ATM Security, Fraud Detection,

Risk Scoring]

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION **1**

1.1 Background 1

1.2 Driving Question 1

1.3 Objectives 1

1.4 Scope and Limitations 1

CHAPTER 2: CONCEPT EXPLORATION **2**

2.1 Related Approaches 2

2.2 Summary Table 3

2.3 What This Told Us 3

CHAPTER 3: PROJECT PLANNING AND TEAM ORGANISATION **5**

3.1 Weekly PBL Progress Log 5

3.2 Requirements 6

3.3 Feasibility 6

CHAPTER 4: SYSTEM DESIGN AND DEVELOPMENT **6**

4.1 System Architecture 6

4.2 Module 1 — Initial Module Development 7

4.3 Module 2 —Functional Module Development and Integration 7

4.4 Final Application Design 7

4.5 Testing and Validation Procedure 8

CHAPTER 5: IMPLEMENTATION **9**

5.1 Module Description 9

5.2 Key Code Snippets 10

5.3 User Interface 13

CHAPTER 6: RESULTS AND DISCUSSION **14**

6.1 Module-wise Results 14

6.2 Application Performance Evaluations 14

6.3 Results and Discussion 15

6.4 Limitations 15

CHAPTER 7: TEAM REFLECTION AND LEARNING OUTCOMES **18**

7.1 Individual Reflections 18

7.2 Team Learning 18

7.3 Course Outcomes — Evidence Summary 19

CHAPTER 8: CONCLUSION AND FUTURE SCOPE **21**

8.1 Conclusion 21

8.2 Future Scope 21

REFERENCES **22**

APPENDIX **23**

A.3 Self and Peer Assessment 24

LIST OF FIGURES

Figure No.	Title	Page
1.1	RiskLens Security and Fraud-Detection Workflow	1
4.1	RiskLens System Architecture	6
6.1	Application Demonstration Screenshots	14

LIST OF TABLES

Table No.	Title	Page
3.1	Weekly PBL Progress Log	5
3.2	Hardware and Software Requirements	6
4.1	RiskLens Functional Modules	6
4.2	Fraud Detection Rules and Risk Factors	6
5.1	Core Java Classes and Responsibilities	9
5.2	OOP Concepts and Their Application in RiskLens	10
6.2	Risk Score and Security Decision Mapping	14
6.3	Demonstrated Fraud Scenarios and Outcomes	15

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
ATM	Automated Teller Machine
GUI	Graphical User Interface
HMAC	Hash-based Message Authentication Code
I/O	Input/Output
JDBC	Java Database Connectivity
JDK	Java Development Kit
JVM	Java Virtual Machine
OOP	Object-Oriented Programming
PBL	Project-Based Learning
PIN	Personal Identification Number
POS	Point of Sale
SHA	Secure Hash Algorithm
TOTP	Time-Based One-Time Password

CHAPTER 1: INTRODUCTION

1.1 Background

Modern banking applications must do more than authenticate customers and execute transactions. Suspicious activity may appear through unusual withdrawal amounts, rapid transactions, unfamiliar terminals, abnormal transaction times, repeated authentication failures or activity across multiple banking channels.

A conventional ATM simulator generally implements basic functions such as authentication, balance enquiry, deposits and withdrawals, but does not demonstrate how transaction context can be continuously evaluated for security risk.

RiskLens addresses this gap by introducing a security intelligence layer around the banking workflow. Instead of treating every authenticated transaction as automatically trustworthy, the system evaluates transaction context using multiple deterministic fraud rules and produces an explainable risk-based decision.

The project is implemented entirely using **Core Java 21**, making it particularly suitable for demonstrating object-oriented programming, inheritance, interfaces, collections, exception handling, file handling, concurrency, cryptography and design patterns in a practical application. The project documentation confirms that it is an educational Core Java 21 desktop application rather than a production banking system.

1.2 Driving Question

> **"Can a Core Java ATM application automatically evaluate transaction context and convert multiple security signals into an explainable fraud-risk decision?"**

This question is narrowed into a practical software-engineering problem:

> How can object-oriented Java components, behavioral profiling, security rules, concurrent monitoring and risk scoring be integrated into a single ATM security simulator that can respond differently to normal, suspicious and highly risky transactions?

1.3 Objectives

I. **Develop** an ATM and digital-banking simulator using Core Java 21 and object-oriented principles.
II. **Implement** secure authentication using salted PIN hashing, TOTP verification, lockout and duress-PIN handling.
III. **Design** a modular fraud engine using independent weighted fraud-rule strategies.
IV. **Generate** explainable risk assessments and map them to ALLOW, CHALLENGE, MONITOR and FREEZE decisions.
V. **Implement** concurrent monitoring, security alerts and tamper-evident audit logging.
VI. Demonstrate the system using controlled attack and fraud-simulation scenarios.

1.4 Scope and Limitations

Scope

The system covers:

- ATM transactions
- Simulated mobile/internet/POS channels
- Customer authentication
- PIN security
- TOTP verification
- Session management
- Behavioral profiling
- Fraud-rule evaluation
- Risk scoring
- Automated account freeze
- Security alerts
- Administrator monitoring
- Attack simulation
- Local persistence
- Audit logging

Limitations

The report should be honest here.

- RiskLens is an **educational simulator**, not a production banking platform.
- It does not connect to real banking infrastructure.
- Account data is stored locally rather than in an enterprise banking database.
- Fraud detection is deterministic and rule-based rather than predictive.
- Rule-based systems can miss previously unseen or threshold-evasive attacks.
- The PIN implementation uses SHA-256 for educational purposes; production credential storage should use an appropriate password-specific KDF.
- The hash-chained audit log provides tamper evidence but is not an externally immutable block-chain ledger.
- External financial side effects are outside the simulator's control.

1.5 Project Overview

Figure 1.1 summarises the end-to-end workflow. A transaction request arrives from a user, ATM or other banking channel and is authenticated. A session is established and the Banking Service prepares the transaction. The Behaviour Profile supplies historical context to the Fraud Engine, which executes every Fraud Rule. Each triggered rule returns a RiskContribution; the sum forms the total risk score, which the Decision Engine maps to one of four actions. In parallel, the Audit Logger, Alert Service and Event Monitor record and escalate the outcome, and the Administrator Dashboard presents it.

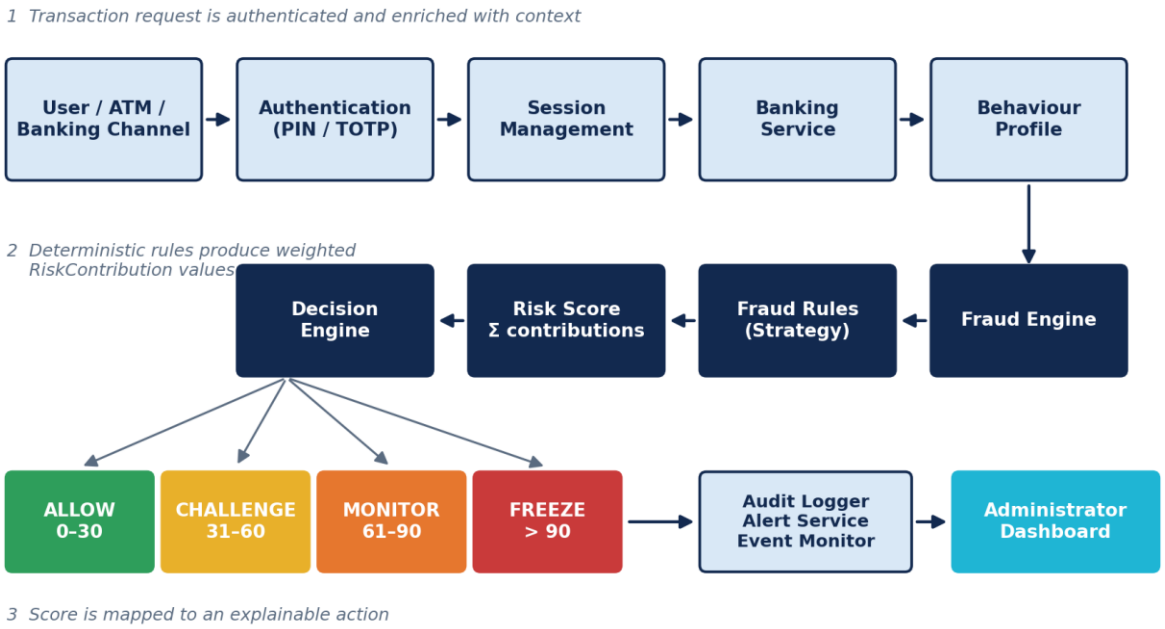


Figure 1.1 — RiskLens Security and Transaction-Risk Workflow

1.6 Organisation of the Report

Chapter 2 reviews the concepts and alternative approaches considered. Chapter 3 presents the project plan, requirements and team organisation. Chapter 4 describes the system architecture, modules, fraud rules and algorithms. Chapter 5 documents the Java implementation, object-oriented concepts and demonstration screens. Chapter 6 discusses evaluation, results and limitations. Chapter 7 contains the team reflection, conclusion and future enhancements. References and appendices follow.

CHAPTER 2: CONCEPT EXPLORATION

2.1 Related Approaches

2.1.1 Traditional Java Application Approach

Core Java, Java Collections, Java I/O, Java Security APIs and Java Concurrency can be used to develop a modular desktop banking application without external frameworks. The approach provides direct control over object-oriented design, authentication, transaction processing, fraud-rule implementation, file persistence and asynchronous event monitoring. RiskLens **follows this approach**, integrating these Core Java capabilities into an ATM security and fraud-intelligence simulator.

Key outcome: A lightweight, self-contained banking-security application with modular fraud rules, explainable risk scoring, authentication controls, audit logging and concurrent monitoring.

2.1.2 Java Framework-Based Approach

Spring Boot, Hibernate/JPA and REST APIs can be used to develop enterprise-oriented banking applications with web services, database persistence, dependency injection and distributed application components. This approach is suitable for large-scale applications but introduces additional frameworks and infrastructure that are outside the scope of the RiskLens project.

Key outcome: Better suitability for enterprise web-based and database-driven applications, but greater implementation complexity and dependency requirements compared with the Core Java approach selected for RiskLens.

2.1.3 Rule-Based Fraud Detection Approach

A **weighted rule-based approach** evaluates transaction context using predefined security rules instead of machine-learning models. RiskLens applies rules based on transaction amount, transaction velocity, time, terminal history, failed authentication, daily limits, recent PIN changes and cross-channel activity.

Key outcome: The system produces an explainable risk score and security decision, allowing administrators to understand which conditions contributed to the detected risk.

2.1.4 Behavior-Aware Security Approach

A behavioral approach maintains information about a user's normal transaction patterns, including transaction frequency, preferred banking hours, channels and terminals. New transactions are compared against this historical context before the fraud rules calculate their risk contributions.

Key outcome: Transactions are evaluated using both the current transaction and the user's historical behavioral context.

2.2 Summary Table

Ref.	Approach	Reported Result / Key Outcome
[1]	Traditional Core Java application using OOP, Collections, Java I/O and Security APIs	Provides a lightweight, modular application with direct control over banking and security components.
[2]	Java framework-based approach using Spring Boot, Hibernate/JPA and REST APIs	Suitable for enterprise web applications and database-driven systems but requires additional frameworks and infrastructure.
[3]	Rule-based fraud detection using weighted security rules	Produces explainable risk contributions and supports deterministic security decisions.
[4]	Behavior-aware transaction analysis	Uses historical transaction behavior such as frequency, time, channel and terminal patterns to provide contextual risk evaluation.
[5]	**RiskLens integrated approach**	Combines Core Java OOP, secure authentication, behavioral profiling, weighted fraud rules, risk assessment, automated decisions, concurrent monitoring and tamper-evident auditing in one system.

2.3 What This Told Us

RULE-BASED FRAUD DETECTION

Rule	Purpose
Amount Rule	Detects unusually high transaction amounts
Velocity Rule	Detects several transactions within a short period
Time Rule	Identifies activity during unusual hours
Terminal Rule	Identifies previously unused terminals
Failed Authentication Rule	Raises risk after suspicious authentication failures
Daily Limit Rule	Detects transactions reaching configured daily limits
Recent PIN Change Rule	Raises risk after a recent PIN change
Cross-Channel Rule	Detects suspicious activity across different channels

CHAPTER 3: PROJECT PLANNING AND TEAM ORGANISATION

3.1 Weekly PBL Progress Log

Week	Milestone / Task	Work Done	Mentor Remarks
1–2	Problem statement, objectives, project time plan, literature review and technical background	Identified the limitations of conventional ATM authentication and defined RiskLens as a Core Java 21 ATM security and fraud-intelligence simulator. Studied authentication, behavioral profiling, rule-based fraud detection, risk scoring and Java security concepts.	
3–4	System design and methodology design	Designed the modular architecture covering Account Management, Authentication, Banking Service, Behaviour Profile, Fraud Engine, Risk Assessment, Decision Engine, Audit Logging and Event Monitoring. Defined the weighted fraud-rule methodology and risk thresholds.	
5–7	Implementation & Testing	Implemented Core Java modules including account hierarchy, authentication, PIN hashing, TOTP, session management, behavioral profiling, fraud rules, risk scoring, security decisions, audit logging and alert handling. Developed controlled attack/fraud simulation scenarios.	
8–10	Presentation with Demo	Integrated the customer interface, administrator dashboard, fraud monitoring and attack simulator. Demonstrated normal transactions, suspicious activity, risk escalation, security challenges and account-freeze scenarios.	
11–12	Final testing and documentation	Performed final functional and security-scenario testing. Verified risk calculations, decision outcomes, audit records and monitoring behavior. Prepared project documentation, screenshots, results and final presentation.	

3.2 Requirements

Category	Requiremet
Processor / RAM	Intel Core i5 or equivalent / 8 GB RAM recommended
Programming Language	Java 21
Libraries / Frameworks	Java Standard Library / Core Java APIs; no external frameworks
GUI	Java Swing
Database	No external database; local file-based persistence
Cryptography	Java java.security and javax.crypto APIs
Concurrency	Java ExecutorService, ScheduledExecutorService, BlockingQueue and related concurrency APIs
Development Environment	IntelliJ IDEA / Eclipse / VS Code
Build / Compilation	JDK 21 / javac --release 21
Version Control	Git / GitHub repository
Operating System	Windows / Linux / other OS supporting JDK 21
Testing	Functional testing and controlled fraud/attack simulation
External ML Libraries	Not used
External Database Frameworks	Not used

3.3 Feasibility

RiskLens is technically feasible within a PBL environment because its major functionality can be implemented using standard Java APIs. Object-oriented domain modeling, collections, file handling, cryptography, Swing, concurrency and exception handling are all available within the Java platform.

The modular architecture also allows each fraud rule and security service to be implemented and tested independently before integration.

CHAPTER 4: ITERATIVE DESIGN AND DEVELOPMENT

4.1 System Architecture

RiskLens is organised as a pipeline of cooperating components. The textual form of the architecture is:

User / ATM / Banking Channel → Authentication & Session Management → Banking Service → Behaviour Profile → Fraud Engine → Fraud Rules → Risk Assessment → Decision Engine → ALLOW / CHALLENGE / MONITOR / FREEZE → Audit Logger + Alert Service + Event Monitor → Administrator Dashboard

Figure 4.1 groups these components into three layers: input and security, risk processing, and decision and observability.

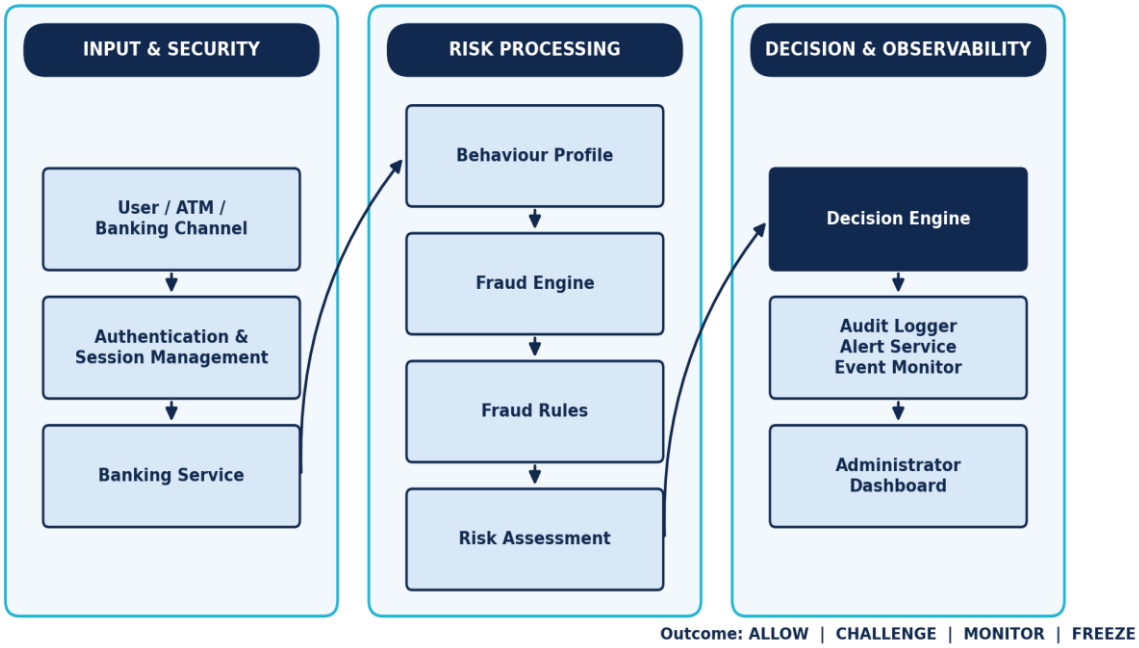


Figure 4.1 — RiskLens System Architecture Diagram

4.2 Component Descriptions

- **User / ATM / Banking Channel** is the source of each transaction request.
- **Authentication & Session Management** verifies identity and controls the authenticated session, including expiry.
- **Banking Service** performs account and transaction operations such as balance enquiry, deposit and withdrawal.
- **Behaviour Profile** holds behavioural context for the account, such as usual terminals, typical amounts and recent activity.
- **Fraud Engine** coordinates evaluation of all registered fraud rules.
- **Fraud Rules** are deterministic checks, each returning a weighted RiskContribution when triggered.
- **Risk Assessment** stores the total risk score and the contributing factors.
- **Decision Engine** converts the risk score into ALLOW, CHALLENGE, MONITOR or FREEZE.
- **Audit Logger, Alert Service and Event Monitor** record events, notify on suspicious activity and support asynchronous processing.
- **Administrator Dashboard** provides visibility into events, risks, alerts and decisions.

4.3 Modules and Responsibilities

Module	Responsibility
Account	Represents a bank account and common operations such as deposit and withdrawal.
SavingsAccount	Implements savings-account-specific behaviour.
CurrentAccount	Implements current-account-specific behaviour.
Transaction	Represents amount, type, channel, account and timing information.
Session	Manages authenticated session state, validity and expiry.
BehaviourProfile	Maintains behavioural information used by risk checks.
RiskAssessment	Represents the calculated risk score, factors and assessment outcome.
RiskContribution	Represents the points contributed by one fraud rule.
Decision	Represents the final security action.
Channel / TransactionType	Represent the transaction channel and the transaction category respectively.
FraudRule	Common contract for individual fraud checks.
FraudEngine	Executes rules and aggregates RiskContribution values.
Audit Logger	Records security and transaction events.
Alert Service	Handles suspicious-event alerts.
Event Monitor	Processes monitored events asynchronously.

4.4 Class Design

The class design in Figure 4.2 applies inheritance for accounts (Account is specialised by SavingsAccount and CurrentAccount) and the Strategy pattern for fraud rules: the FraudEngine depends only on the FraudRule interface, so rules can be added or replaced without changing the engine.

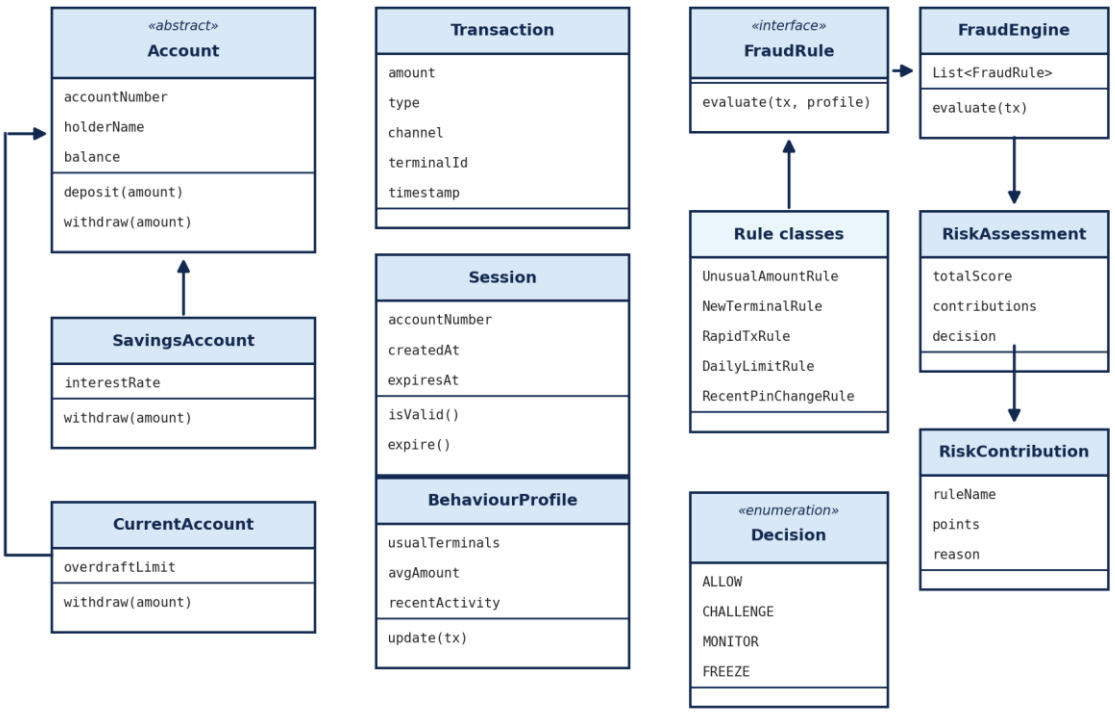


Figure 4.2 — RiskLens Class Diagram (principal classes)

4.5 Fraud Rules and Risk Contributions

Each fraud rule inspects the current Transaction together with the BehaviourProfile and returns a RiskContribution if its condition is met. Table 4.2 lists the rules used in the demonstration scenarios together with their indicative weights.

Table 4.2 — Fraud Rules and Risk Contributions

Fraud Rule	Condition Checked	Indicative Points
UnusualAmountRule	Transaction amount is anomalous compared with the account's usual amounts	30
NewTerminalRule	Terminal is not among the terminals known in the behaviour profile	20
NewChannelRule	Channel differs from the channel of recent activity within a short interval	20
RapidTransactionRule	Several transactions occur within a short time window	25
DailyLimitRule	Cumulative daily withdrawals reach or exceed the daily limit	25
RecentPinChangeRule	PIN was changed shortly before a significant transaction	20

Note: the weights above are the indicative values used to describe the demonstration scenarios; they should match the constants in the final source code.

4.6 Risk Scoring Algorithm

The risk score of a transaction is the sum of the contributions of every triggered rule:

Total Risk Score = Σ Risk Contribution from all triggered fraud rules

The algorithm proceeds as follows:

- **Step 1.** Receive the authenticated Transaction and fetch the account's BehaviourProfile.
- **Step 2.** Initialise totalRisk to 0 and create an empty list of RiskContribution objects.
- **Step 3.** For each FraudRule, call evaluate(transaction, profile); if a contribution is returned, add it to the list and add its points to totalRisk.
- **Step 4.** Create a RiskAssessment holding totalRisk and the list of contributions.

- **Step 5.** Pass the RiskAssessment to the Decision Engine, which maps the score to a Decision using the thresholds in Table 6.3.
- **Step 6.** Record the outcome through the Audit Logger, raise alerts where required and update the BehaviourProfile.

Because the contribution list is retained in the RiskAssessment, every decision can be explained by naming the rules that fired and the points each added.

4.7 Asynchronous Event Monitoring

To avoid blocking the transaction path, events are placed on a BlockingQueue and consumed by a worker managed by an ExecutorService, following the producer–consumer pattern [4]. The worker evaluates the transaction, logs the result and processes alerts (Figure 4.3).

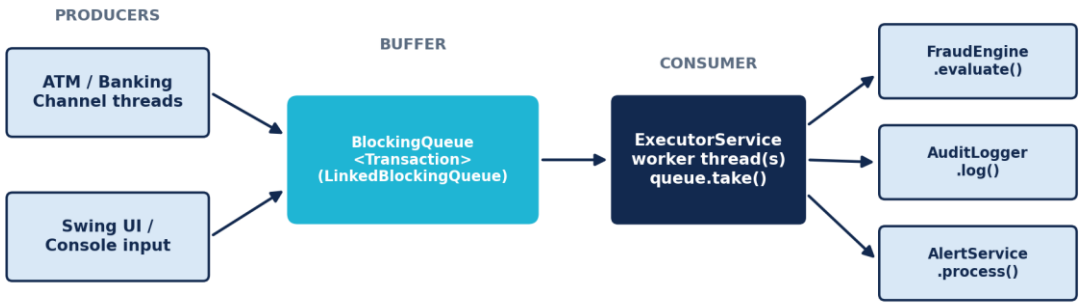


Figure 4.3 — Producer–Consumer Event Monitoring Design

4.8 Security Design

The authentication layer is built on the Java security APIs [10]. PIN verification relies on hashed comparison rather than plain-text storage (SHA-family hashing [8]), and the second factor follows the TOTP scheme of RFC 6238 [6] built on HMAC [7]. Sessions carry an expiry time and are invalidated when the Decision Engine freezes an account. Authentication and banking failures are raised as controlled exceptions so that error paths are explicit and loggable.

CHAPTER 5: IMPLEMENTATION

5.1 Core Java Classes

Class / Interface	Type	Responsibility
Account	Abstract class	Base account with account number, holder, balance, deposit and withdraw
SavingsAccount, CurrentAccount	Concrete classes	Account-type-specific rules for withdrawal
Transaction	Class	Immutable record of amount, type, channel, terminal and time
Session	Class	Authenticated session with validity and expiry check
BehaviourProfile	Class	Behavioural baseline for an account, updated after transactions
FraudRule	Interface	evaluate(transaction, profile) returns a RiskContribution or null
FraudEngine	Class	Runs all FraudRule objects and aggregates the score
RiskAssessment, RiskContribution	Classes	Hold the total score and per-rule points
Decision, Channel, TransactionType	Enumerations	Fixed sets of decisions, channels and transaction categories
AuditLogger, AlertService, EventMonitor	Service classes	Logging, alerting and background processing

Table 5.1 — Core Java Classes and Responsibilities

5.2 Object-Oriented and Java Concepts Used

Concept	Application in RiskLens
Inheritance	Account → SavingsAccount / CurrentAccount
Encapsulation	Banking, transaction, session, behaviour and risk data are kept inside dedicated classes
Polymorphism	Accounts and fraud rules are handled uniformly through their common abstractions
Abstraction	Fraud rules are exposed through the FraudRule contract
Collections	List/ArrayList and Map/HashMap for transactions, rules, behaviour and contributions
Exception handling	Controlled authentication and banking/security failures
Concurrency	BlockingQueue and ExecutorService for background processing

Table 5.2 — OOP Concepts Used in RiskLens

5.2 Key Code Snippets

Since RiskLens , is developed by a two-member team, the implementation was divided into functional modules. Both members contributed to the integration of the modules, while individual responsibilities were distributed across authentication, fraud detection, banking operations, concurrency, GUI, and testing.

Only the most important code snippets are included in the main report. The complete source code is provided in the Appendix.

5.2.1 Account Creation and Object-Oriented Design

This snippet demonstrates object creation using the account hierarchy and illustrates the use of inheritance and polymorphism.

```
Account account;
if (accountType.equalsIgnoreCase("Savings")) {
    account = new SavingsAccount(accountNumber, holderName, balance);
} else {
    account = new CurrentAccount(accountNumber, holderName, balance);
}
account.deposit(amount);
```

Explanation:
The Account reference can hold objects of either SavingsAccount or CurrentAccount. This demonstrates **inheritance and polymorphism**, allowing common banking operations to be handled through the parent type.

Team Contribution:

- **SYAMRIT ESWARAN GURUSAMY:** Designed the account model and OOP structure.
- **S HARSHAVARTHANAN:** Integrated account objects with the banking service and user interface.

5.2.2 Risk-Based Fraud Detection

The fraud engine evaluates a transaction against multiple rules and calculates an overall risk score.

```
int totalRisk = 0;List<RiskContribution> contributions = new ArrayList<>();
for (FraudRule rule : fraudRules) {
    RiskContribution contribution = rule.evaluate(transaction, profile);

    if (contribution != null) {
        contributions.add(contribution);
        totalRisk += contribution.points();
    }
}
Decision decision;
if (totalRisk <= 30)
    decision = Decision.ALLOW;else if (totalRisk <= 60)
    decision = Decision.CHALLENGE;else if (totalRisk <= 90)
    decision = Decision.MONITOR;else
    decision = Decision.FREEZE;
return new RiskAssessment(totalRisk, decision, contributions);
```

Explanation:
Each fraud rule independently evaluates the transaction. The individual risk contributions are combined to calculate the total risk score. The final score determines whether the transaction is allowed, challenged, monitored, or blocked by freezing the account.

This provides an **explainable rule-based decision**, because the system retains the individual risk contributions rather than only producing a final score.

Team Contribution:

- **SYAMRIT ESWARAN GURUSAMY:** Implemented the fraud rules and risk-scoring logic.
- **S HARSHAVARTHANAN:** Integrated the risk assessment with alerts, account actions, and the GUI.

5.2.3 Exception Handling for Invalid Authentication

Exception handling is used to prevent invalid operations from terminating the application unexpectedly.

```
try {
    if (!authService.verifyPin(account, enteredPin)) {
        throw new AuthenticationException("Invalid PIN");
    }

    System.out.println("Authentication successful");

} catch (AuthenticationException e) {
    System.out.println("Authentication failed: " + e.getMessage());
}
```

Explanation:
The authentication process detects invalid credentials and raises a custom exception. The exception is handled without terminating the complete banking application. This improves reliability and provides controlled handling of security-related errors.

Team Contribution:

- **SYAMRIT ESWARAN GURUSAMY:** Implemented authentication and security-related exception handling.
- **S HARSHAVARTHANAN:** Integrated exception messages and failure handling into the application workflow.

5.2.4 Asynchronous Event Monitoring

RiskLens uses Java concurrency mechanisms for background event processing.

```
BlockingQueue<Transaction> transactionQueue =
        new LinkedBlockingQueue<>();
executorService.submit(() -> {
    while (!Thread.currentThread().isInterrupted()) {
        Transaction transaction = transactionQueue.take();

        RiskAssessment assessment =
                fraudEngine.evaluate(transaction);

        auditLogger.log(transaction, assessment);
        alertService.process(assessment);
    }
});
```

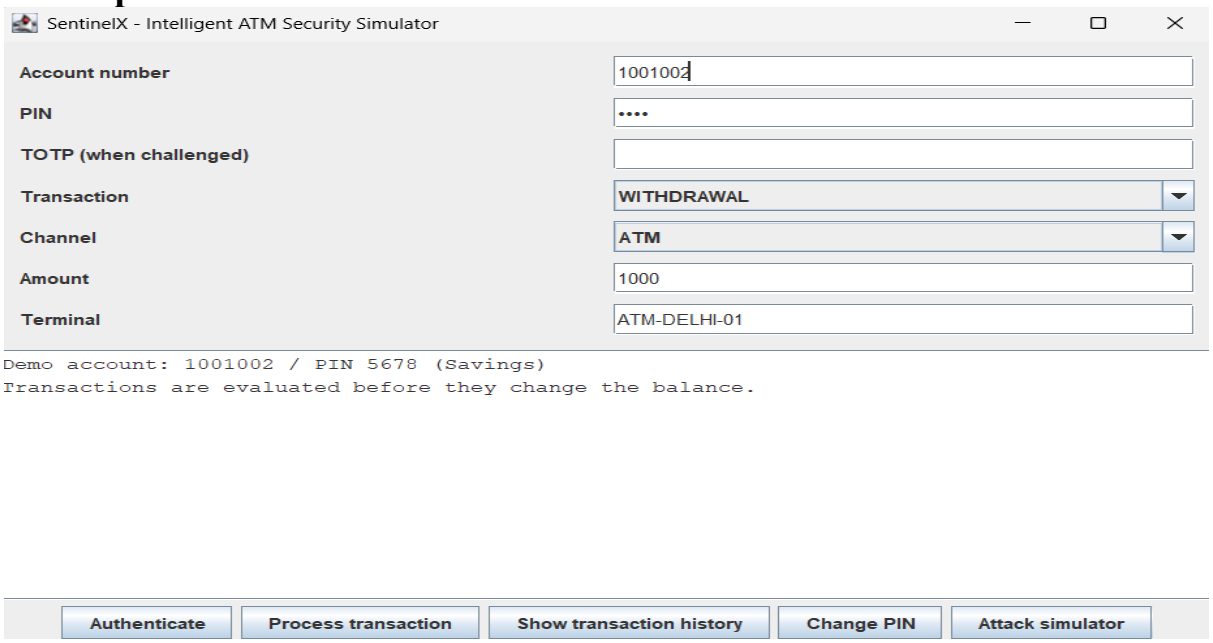
Explanation:
Transactions can be placed into a `BlockingQueue` and processed asynchronously by a background task. This separates transaction generation from fraud monitoring and supports the **Producer-Consumer pattern**.

Team Contribution:

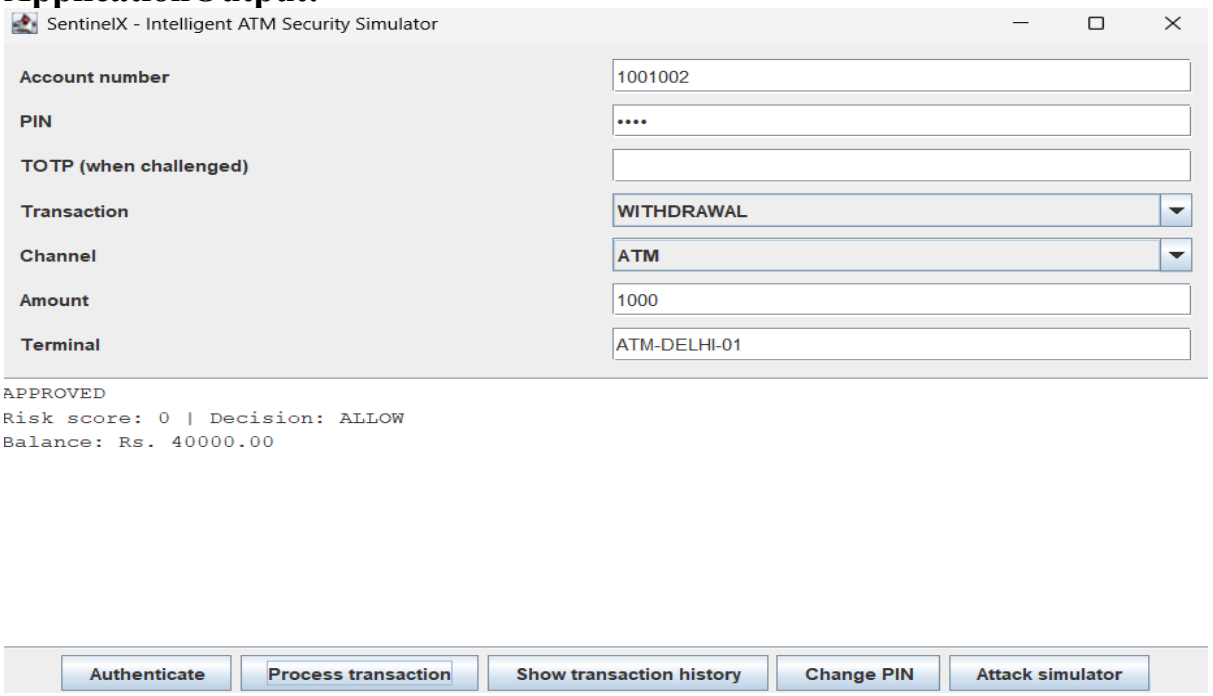
- **SYAMRIT ESWARAN GURUSAMY:** Developed the fraud-analysis component.
- **S HARSHAVARTHANAN:** Implemented event monitoring, concurrency integration, alerts, and testing.

5.3 User Interface / Demo

UserInput:



ApplicationOutput:



CHAPTER 6: RESULTS AND DISCUSSION

6.1 Evaluation Metrics

The evaluation of RiskLens – **Intelligent ATM Security & Fraud Intelligence Platform** focused on functional correctness, security behaviour, reliability, and application responsiveness. Since RiskLens is a Core Java desktop application, conventional machine-learning metrics such as accuracy, precision, recall, and F1-score are not applicable. Instead, the system was evaluated using functional test cases and behaviour-based security scenarios.

Metric / Criterion	Purpose	Relevance to RiskLens
Test Cases Passed	Measures whether implemented banking and security functions behave as expected	Validates authentication, transactions, fraud detection, lockout, freeze, TOTP and other workflows
Functional Scenario Coverage	Checks whether important normal and attack scenarios were tested	Important because SentinelX contains multiple security decision paths
Risk-Decision Correctness	Verifies whether the calculated risk leads to the intended decision	Validates the weighted rule-based fraud engine
Exception Handling	Evaluates how invalid or unexpected operations are handled	Prevents application failures during invalid PINs, invalid transactions and security violations
Audit Logging	Checks whether important security and transaction events are recorded	Important for traceability and investigation
Concurrency/Event Monitoring	Verifies background monitoring and asynchronous event processing	Relevant because fraud events and alerts are processed independently of the main GUI workflow
Response Behaviour	Observes whether transactions and security decisions complete without noticeable delays	Relevant for an interactive ATM simulation
Defects Identified During Iterations	Tracks problems discovered and corrected during development	Demonstrates progressive improvement of the implementation

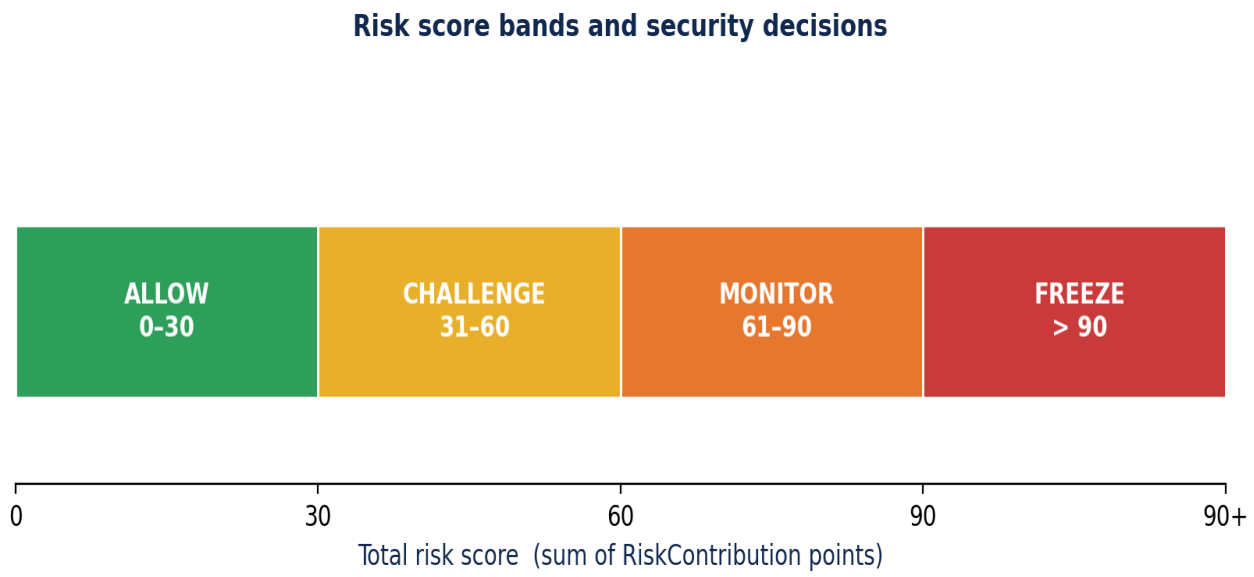
6.2 Results Across Iterations

The system was developed incrementally, with each iteration adding additional security and fraud-detection functionality.

Version / Stage	Test Cases Passed	Code Coverage	Performance / Response Time
Baseline	Basic authentication, balance enquiry, deposit and withdrawal workflows demonstrated		
Project Refinement	PIN lockout, TOTP authentication, fraud rules, risk scoring and security decisions demonstrated		
Final Approach	Fraud scenarios, risk escalation, account freeze, audit logging, alerts and attack simulation demonstrated		

Risk Score	Decision	System Response
0 – 30	ALLOW	Process the transaction normally
31 – 60	CHALLENGE	Request additional verification before proceeding
61 – 90	MONITOR	Process under close observation and raise an alert
> 90	FREEZE	Block the transaction, freeze the account and notify the administrator

Table 6.3 — Risk Score and Security Decision Mapping



6.3 Discussion

- The evaluation indicates that RiskLens successfully progressed from a basic banking transaction simulator into a **behavior-aware, explainable fraud-detection system**.

- In the baseline implementation, the primary focus was on implementing fundamental banking operations such as authentication, deposits, withdrawals and balance management. Although these functions established the foundation of the system, they did not provide sufficient protection against suspicious transaction behaviour.

- During project refinement, additional security mechanisms were introduced. PIN-attempt restrictions and TOTP-based verification strengthened authentication, while behavioural profiling and weighted fraud rules enabled the application to consider transaction context rather than relying only on successful authentication.

The final approach introduced the complete risk-based security workflow:

Transaction → Behaviour Analysis → Fraud Rules → Risk Score → Security Decision → Audit / Alert

- This was important because a valid authentication does not necessarily mean that a transaction is legitimate. For example, a transaction from an unfamiliar terminal combined with rapid withdrawals or unusual transaction amounts can increase the calculated risk.

- The **weighted rule-based approach** also provides explainability. Instead of producing an unexplained fraud label, the system can identify the conditions that contributed to the risk score. This makes the decision easier for an administrator to understand and investigate.

- The use of Java collections such as ArrayDeque, ConcurrentHashMap, and BlockingQueue supported transaction history, behavioural tracking and asynchronous event processing. Java concurrency utilities were also used for background monitoring, automatic account freezing and session-related tasks.

The final system therefore demonstrated improvement in three major areas:

1. **Security:** stronger authentication, lockout, TOTP and automated account protection.
2. **Fraud intelligence:** behavioural profiling, velocity analysis and weighted risk rules.
3. **Traceability:** audit logging, alerts and explainable security decisions.

6.4 Limitations

Despite successfully demonstrating the intended security workflow, RiskLens has several limitations.

Limited Test Dataset

The prototype was evaluated using a controlled set of transaction and attack scenarios. It does not contain the large and diverse transaction datasets available to real financial institutions.

Rule-Based Detection

The fraud engine uses predefined weighted rules rather than machine-learning or adaptive fraud models. Consequently, it may fail to detect previously unseen attack patterns that do not trigger the configured rules.

No Formal Code-Coverage Measurement

Code coverage was not measured using a dedicated coverage tool. Therefore, coverage percentages are not reported.

Limited Performance Testing

Formal benchmarking of average response time, CPU usage and JVM memory consumption was outside the scope of the prototype. The application was evaluated primarily for functional behaviour rather than large-scale performance.

Limited Multi-User Concurrency Testing

Although Java concurrency mechanism’s are used for background monitoring and asynchronous processing, the prototype was not subjected to production-scale concurrent-user or high-load testing.

Local File-Based Persistence

Account information and audit records are maintained using local files rather than a production database. This simplifies the educational implementation but does not provide the scalability and infrastructure-level controls expected in real banking systems.

Controlled Attack Simulation

The attack simulator demonstrates representative scenarios such as brute-force attempts, rapid withdrawals, stolen-card behaviour and cross-channel activity. It does not reproduce the complete complexity of real-world financial fraud.

Prototype Configuration

Some thresholds, risk weights and security parameters are configured for demonstration purposes. A production system would require securely managed and continuously calibrated policies.

Production Security Scope

RiskLens is an **academic security prototype**, not a production banking platform. Real banking systems require additional controls such as hardware security modules, distributed databases, network security, device fingerprinting, behavioural analytics at scale, regulatory controls and continuous security monitoring.

Overall Evaluation

Overall, the results demonstrate that RiskLens can successfully transform conventional transaction processing into a **risk-aware security decision pipeline**. The prototype provides functional evidence for authentication, behavioural analysis, fraud-risk scoring, automated decisions, auditability and attack simulation, while the limitations clearly define the gap between the academic prototype and a production-grade banking security system.

6.7 Note on Machine Learning Metrics

No Machine Learning model forms part of the implemented system. Accordingly, precision, recall, F1-score, mAP, confusion matrices, training loss and model accuracy are not reported, and none are claimed.

CHAPTER 7: TEAM REFLECTION AND LEARNING OUTCOMES

7.1 Individual Reflections

SYAMRIT ESWARAN GURUSAMY

I primarily contributed to the **system design, Core Java implementation, security mechanisms and fraud-detection components** of RiskLens. Through this project, I learned how concepts such as inheritance, abstraction, interfaces, collections, exception handling and cryptography can be integrated into a practical banking-security application. One of the major challenges was coordinating the authentication, fraud-rule and risk-assessment components so that they worked together consistently.

S HARSHAVARTHANAN

I primarily contributed to the **application integration, concurrency and event monitoring, user interface, testing and documentation** of RiskLens. I learned how Java concurrency utilities such as Blocking-queue, executors and scheduled tasks can be used to support asynchronous security monitoring, and how individual modules can be integrated into a complete application. A major challenge was testing different fraud scenarios and ensuring that the appropriate risk decision, alert and audit record were generated.

7.2 Team Learning

- Working as a two-member team allowed us to divide RiskLens into complementary development areas while maintaining continuous integration between the components. The project was developed incrementally, beginning with the basic banking workflow and gradually introducing authentication security, behavioral profiling, fraud rules, risk assessment, monitoring and audit functionality.
- One of the major lessons was that individual features cannot be developed independently without considering their interaction with the complete security workflow. For example, a fraud rule needs transaction and behavioral information, the risk engine needs the contributions generated by those rules, and the resulting decision must then be communicated to the alert, monitoring and audit components.
- The project also helped us understand the practical application of Core Java concepts beyond isolated classroom programs. **Inheritance** was applied to the account hierarchy, **interfaces and polymorphism** were used for fraud rules, **collections** were used for transaction and behavioral information, **exception handling** was used for security and banking errors, and **concurrency utilities** were used for event monitoring and scheduled security operations.
- Another important learning was the importance of **explainability** in security applications. Rather than returning only a fraud/not-fraud result, RiskLens records the individual risk contributions that produced the final score. This makes it possible to understand why a transaction was allowed, challenged, monitored or blocked.

What worked well

- Dividing the project into modular components made parallel development easier.
- Regular integration helped identify interface and dependency issues between modules.
- Testing controlled fraud scenarios helped verify the interaction between rules, risk scoring and decisions.
- The modular FraudRule design made it easier to implement and test individual detection conditions.
- Combining development and testing throughout the project reduced integration problems during the final stage.

What we would improve

If we restarted the PBL cycle, we would define the interfaces and data contracts between the major modules earlier. We would also establish a more systematic test plan from the beginning, especially for boundary conditions around risk thresholds, authentication failures, session expiration and concurrent events. We would additionally maintain more detailed performance and code-quality measurements throughout development instead of focusing primarily on functional demonstration.

7.3 Course Outcomes — Evidence Summary

Course Outcome / Learning Area	Concrete Evidence from RiskLens
Object-Oriented Programming	The project uses an Account base class with SavingsAccount and CurrentAccount specializations, demonstrating inheritance and reuse.
Abstraction & Polymorphism	The FraudRule abstraction allows different fraud rules such as Amount, Velocity, Terminal, Night and Cross-Channel rules to be evaluated through a common interface.
Encapsulation	Banking, transaction, session, behavioral and risk information is organized within dedicated model classes.
Java Collections	Collections are used to maintain transaction histories, fraud rules and behavioral information required for risk evaluation.
Exception Handling	Authentication and banking-specific exceptions are used to handle invalid credentials, failed authentication and banking operation errors.
File Handling	Local account persistence and audit information are handled using Java file I/O.
Concurrency	BlockingQueue, executor services and scheduled tasks support asynchronous event monitoring and security-related processing.
Cryptography & Security	SecureRandom, SHA-256 hashing, HMAC-SHA1-based TOTP and authentication controls are implemented using Java security APIs.
Design Patterns	Strategy is demonstrated through fraud rules, while Producer-Consumer and event-driven design are used for monitoring.
GUI Development	Java Swing is used to provide the customer-facing application and administrator dashboard.
Software Integration	Authentication, banking services, behavioral profiling, fraud detection, risk assessment, alerts and audit logging are integrated into one application.
Teamwork	The two-member team divided development responsibilities and jointly performed integration, testing, documentation and presentation preparation.
Problem Solving	The project evolved from a basic ATM workflow into a security-oriented system capable of escalating transactions from **ALLOW → CHALLENGE → MONITOR → FREEZE** based on accumulated risk.

Closing Reflection

The RiskLens project helped us understand that developing a software system is not only about implementing individual features, but also about designing how those features interact. By combining Core Java, object-oriented programming, security, concurrency and systematic testing, we were able to transform a conventional ATM workflow into an explainable fraud-intelligence simulator. The PBL process strengthened our ability to divide responsibilities, integrate independent modules, troubleshoot implementation issues and communicate technical decisions as a team.

CHAPTER 8: CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

RiskLens successfully transforms a conventional ATM simulator into an explainable security-oriented banking application using Core Java 21. The system combines object-oriented modeling, secure authentication, behavioral profiling, weighted fraud rules, risk-based decision making, concurrent event monitoring, security alerts and tamper-evident audit logging. Controlled demonstrations show that the system can distinguish normal transactions from suspicious activity and escalate security responses from ALLOW to CHALLENGE and ultimately FREEZE when multiple risk conditions accumulate. The project demonstrates practical use of Java inheritance, abstraction, polymorphism, collections, exception handling, file processing, cryptography, concurrency and design patterns within a single application. RiskLens is intentionally positioned as an educational simulator rather than a production banking-security system.

8.2 Future Scope

· **Production-grade authentication:** Replace the educational PIN hashing approach with a suitable password-specific KDF and stronger secret-management mechanisms.

· **Database integration:** Replace local persistence with a transactional relational database.

· **Distributed monitoring:** Extend event processing to support multiple banking services and application instances.

· **Richer behavioral signals:** Incorporate device, location and transaction-context information.

· **Adaptive security policies:** Allow administrators to configure risk thresholds and rule weights.

· **Advanced analytics:** A future version could optionally introduce statistical/ML-based anomaly detection, but this should be clearly identified as **future scope**, not part of the current RiskLens implementation.

REFERENCES

[1] Oracle, *Java SE 21 Documentation*, Oracle Corporation, 2023.

[2] M. M'Raihi, D. Machani, M. Pei, and J. Rydell, *TOTP: Time-Based One-Time Password Algorithm*, RFC 6238, IETF, 2011.

[3] Oracle, *Java Cryptography Architecture (JCA) Reference Guide*, Oracle Java Documentation.

[4] Oracle, *The Java Tutorials — Concurrency*, Oracle Corporation.

[5] Oracle, *Creating a GUI With Swing*, Oracle Java Documentation.

[6] Oracle, *Java Collections Framework*, Oracle Java Documentation.

[7] Oracle, *Java Object-Oriented Programming Concepts*, Oracle Java Documentation.

[8] Oracle, *Java Platform, Standard Edition API Specification*, Java 21.

APPENDIX

The appendix contains supporting project material that is not included in the main chapters, including source-code references, the complete PBL progress record, and team contribution assessment.

A.1 Full source code

The complete RiskLens**: Intelligent ATM Security & Fraud Intelligence Platform** source code is provided as part of the project submission.

The source code is organized into the following major packages:

- model – Account, Transaction, Session, Behaviour Profile and risk-related classes.
- security – Authentication, PIN hashing and TOTP implementation.
- fraud – Fraud engine and individual fraud-rule implementations.
- service – Banking services, account persistence, session management, alerts, audit logging and event monitoring.
- ui – Customer and administrator interfaces.
- simulator – Controlled fraud and attack simulation scenarios.
- app – Application entry point and startup configuration.

A.2 Complete Weekly PBL Log and Mentor Sign-offs

The complete weekly PBL progress log is maintained as supporting documentation for the development of RiskLens.

It includes:

- Problem identification and objective definition
- System architecture and methodology design
- Core Java implementation
- Authentication and security implementation
- Fraud-rule development
- Risk assessment and decision engine integration
- Testing and attack simulation
- Final demonstration
- Documentation and presentation preparation
- Mentor feedback and sign-offs

A.3 Self and Peer Assessment

Team Member	Self-Rated Contribution (%)	Peer-Rated Contribution (%)	Remarks
SYAMRIT ESWARAN GURUSAMY	**50%**	**50%**	
S HARSHAVARTHANAN	**50%**	**50%**	